

Ontology-Constrained Vector Retrieval

Joseph Natter

Department of Computer Science
Kennesaw State University
Marietta, GA, USA
jnatter@students.kennesaw.edu

Nicholas Hodge

Department of Computer Science
Kennesaw State University
Marietta, GA, USA
nhodge8@students.kennesaw.edu

Abstract—Vector databases have emerged as the foundational infrastructure for semantic retrieval, offering scalable approximate nearest-neighbor search over high-dimensional embedding representations. However, by organizing data through similarity alone, these systems provide limited guarantees about the correctness of retrieved results. In many real-world applications, valid results must conform to structural constraints.

We present an ontology-constrained vector retrieval framework that integrates symbolic ontological structure directly into the retrieval process. By deriving type constraints from an RDF/OWL knowledge graph and enforcing them as pre-hoc filters prior to similarity ranking, our approach brings formal ontological structure to bear on vector search without modifying the underlying database. We evaluate our method on NASA’s Air Traffic Management ontology (ATMONT) which includes 2.93 million RDF triples, 36,655 indexed entities), the Brick building schema, and a selection of the DBpedia ontology, across two embedding models. Pre-hoc ontology-constrained retrieval achieves perfect type validity ($\text{valid@5} = 1.00$) across all configurations, compared to 0.66-0.75 for unconstrained retrieval, with only modest latency overhead. Post-hoc filtering requires examining up to 2,000 candidates to approach comparable validity, and fails to reach full validity for weaker embedding models. These results demonstrate that ontological structure, when foregrounded in the retrieval process, provides a practical and effective mechanism for enforcing semantic correctness in vector search systems.

Index Terms—ontology, vectors, databases

I. INTRODUCTION

Database management systems have long served as the foundational infrastructure for data-intensive applications, providing efficient mechanisms for storing, indexing, and querying large collections of structured data. However, the increasing reliance of modern machine learning pipelines on high-dimensional embedding representations has exposed fundamental limitations of traditional relational and key-value data models, which are not designed to support similarity-based retrieval in continuous vector spaces.

Vector databases have emerged to address this gap by offering scalable approximate nearest-neighbor search and vector-native indexing structures, enabling efficient retrieval over large embedding collections. As a result, vector database systems have seen rapid adoption across a range of machine learning workloads, including semantic search, recommendation systems, and representation-based retrieval pipelines, and are now widely recognized as a distinct and important class of data management systems [1], [2]. These systems now underpin a wide range of workloads beyond natural language pro-

cessing, including recommendation systems, computer vision, and scientific and sensor data management. By organizing data primarily through geometric similarity in embedding space, vector databases offer scalable approximate nearest-neighbor search, but provide limited guarantees about the semantic correctness of retrieved results.

In many real-world applications, retrieved results must satisfy domain-specific semantic constraints, such as type compatibility, hierarchical relationships, or disjointness conditions, which are naturally expressed through ontologies. Existing vector retrieval systems handle such constraints through flat metadata or predicate filters applied at pre-, post-, or runtime stages of filtered approximate nearest neighbor search. However, the choice of stage involves a critical trade-off in retrieval quality and efficiency. Generic post-hoc filtering strategies have been shown to suffer from significant recall degradation, making them unsuitable for high-precision applications [3]. Conversely, while pre-hoc keyword filtering can preserve recall, it faces architectural limitations in graph-based indexes and therefore keyword pre-filtering in these structures does not reduce the search space or improve latency as effectively as it does in partition-based indexes [3]. Moreover, such metadata filters enforce only explicitly encoded predicates, not the richer type relationships implied by ontology class hierarchies. Consequently, as highlighted by recent work, integrating structured constraints with approximate vector search remains a nontrivial efficiency and recall problem [4], [5].

This paper argues that semantic constraints should be foregrounded in the process of vector retrieval and presents an ontology-constrained vector retrieval framework that integrates ontology-derived class hierarchy constraints into retrieval, rather than as an external filtering step.

Our contributions are: (i) a formalization of ontology-constrained vector retrieval, (ii) a practical implementation using pre-hoc filtering in a vector database, and (iii) an empirical evaluation demonstrating improved semantic validity with minimal overhead.

Our implementation and evaluation code are publicly available at <https://github.com/nicholashodge/ocvd>.

II. RELATED WORK

A. Ontology

The term ontology comes from philosophy, where it is the “science of what is, of the kinds and structures of objects,

properties, events, processes, and relations in every area of reality” [6].

A natural question is why such representations are useful.

The first answer to this motivation is that automated natural language is naturally “limited by the ambiguity of language” [7]. Any formal framework for conceptualization, especially with the wide scope of “everything” is a crucial and ambitious step for the field of knowledge representation.

The second answer to why we would want to represent data using ontologies is that it is the natural next step for “research in computer science, artificial intelligence, philosophy, library science, and linguistics,” as these fields have already been working to create “standard descriptions and terminology for the entities and events that make up our world” [7].

In other words, ontological structuring in a formal framework will be the ideal formatting for general knowledge representation as it is not limited by natural language, and is a natural fit for many fields concerned with knowledge representation.

B. Ontology Databases

Modern ontology databases are typically built upon the Resource Description Framework (RDF), a graph-based data model in which all information is represented as triples of the form (*subject, predicate, object*). Collectively, RDF triples form a labeled directed graph and are queried using SPARQL, a declarative pattern-matching language whose execution resembles relational join processing over graph-structured data.

Efficient storage and evaluation of large RDF repositories has been extensively studied. Systems such as RDF-3X [8] demonstrate that schema-free graph data can be managed at scale using relational-style indexing and cost-based optimization. RDF-3X employs a single triples table architecture together with exhaustive indexing over all six permutations of subject, predicate, and object, enabling searches by any of the RDF attributes, pairs, or triplets. This work shows that RDF data can be stored compactly and queried efficiently without requiring workload-specific physical design tuning.

While RDF defines a flexible representation for graph data, it does not by itself impose rich semantic constraints. Formal semantics are introduced through RDF Schema (RDFS) and the Web Ontology Language (OWL) [9], [10]. OWL is grounded in description logics (DL) and enables the specification of class hierarchies, property characteristics, disjointness axioms, and domain and range restrictions under formally defined semantics. In ontology-based systems, the ontology defines a conceptual schema layer (TBox), while instance data populates this structure as assertions (ABox).

Ontology databases therefore combine: (i) a triple-based physical storage layer (RDF), (ii) a formally defined logical vocabulary and reasoning layer (OWL), and (iii) a query interface (SPARQL) capable of retrieving both asserted and entailed facts.

However, despite their expressive semantic guarantees, RDF/OWL systems are optimized for symbolic reasoning and graph-pattern matching rather than similarity-based retrieval

in high-dimensional embedding spaces. SPARQL is based on matching graph patterns over RDF graphs, while RDF and OWL provide formal semantic interpretations and entailment regimes for deriving additional logical statements [9]–[12]. In contrast, vector databases are designed around nearest-neighbor and approximate nearest-neighbor search over dense vector representations [1], [13]. RDF/OWL systems therefore provide logical validity under formal semantics, but do not natively support geometric similarity search or approximate nearest-neighbor retrieval. This limitation motivates the integration of ontological reasoning with vector-native database systems.

C. Vector Database

Vector databases (VDBs) are a popular solution to the problem of processing and handling copious amounts of data in different modalities. VDBs encode various forms of data into vectors and store them in high dimensional formats; the payoff is that vectors are able to represent not only the data itself but the semantic and contextual information behind it. VDBs are behind many of the applications in both specifically large language models (LLMs) and greater artificial intelligence (AI) overall, due to their efficiency, storage capability, and scalability [13].

D. Converting an OWL Document to a Relational Database

There has been a direct attempt to design an algorithm that can convert a given OWL document to a relational database, dubbed *OWL2DB*, designed by Gali, et. al in [14]. Motivated by the success behind SQL and relational database systems, this work was done in direct contrast to a popular alternative approach, which is to convert OWL to extensible markup language (XML).

There were two major parts of their work: the *OWL2DB* algorithm and the mapping system.

The mapping system is responsible for the mapping between OWL and the relational schema, and is broken into three parts: Ontology Model, Document Manager, and Ontology Resource Manager.

The ontology model takes OWL documents and outputs a comprehensive ontology model based on the type of OWL document provided: OWL DL, OWL full, or OWL Lite. The document manager can process multiple OWL documents, and tracks where statements from the OWL came from. The ontology reasoner is the final part of the process, responsible for using the resource description framework [15] to each resource from the ontology.

The second part of the pipeline is the *OWL2DB* algorithm. The algorithm accepts an OWL ontology and converts it into a relational database schema by first parsing the document to identify root classes and traversing their subclass hierarchies to determine depth. As the class graph is explored, subclass depth is tracked, and when a depth of three is reached, relation tables are generated for higher-level subclasses, using class attributes and instances as column names. This process is repeated throughout the hierarchy to collect table names and

column definitions, while ensuring that union, complement, and disjoint subclasses at the same level are handled consistently, with disjoint classes mapped to separate relations. The ontology is then parsed again to extract instance values, after which the database tables are created and populated.

This work is relevant for two primary reasons: 1) our dataset is built using OWL full documents, which can be converted using the OWL2DB algorithm and 2) the *ontology modeler* portion of the mapping system could be used in conjunction with vector embeddings, accomplishing our goal of placing the ontology into a pre-hoc or ad-hoc solution.

Ultimately, the “conclusions” derived by the authors lacked comprehensive benchmarking. We extend this line of work by evaluating retrieval performance using quantitative metrics.

III. PROBLEM SETTING

We consider the problem of semantic retrieval over a collection of entities derived from an ontology. Let $\mathcal{E}$ denote a set of entities extracted from an ontology-backed dataset, where each entity $e \in \mathcal{E}$ is associated with: (i) a textual representation t_e , and (ii) a set of ontology types $\mathcal{T}_e$ defined by class assertions and their transitive closure under `rdfs:subClassOf`.

Given a natural language query q , standard vector retrieval embeds both q and each t_e into a shared embedding space and returns the top- k entities according to a similarity function:

$$\text{Top}_k(q) = \arg \max_{e \in \mathcal{E}} \text{sim}(q, t_e)$$

While this approach captures semantic similarity, it does not enforce that retrieved entities satisfy domain-specific constraints. In practice, this leads to results that are semantically related to the query but violate expected type constraints (e.g., returning airports when querying for route segments).

We therefore define ontology-constrained retrieval as the problem of retrieving entities that are both semantically relevant and type-consistent with a target ontology class (or set of classes) $\mathcal{C}$. Formally, we seek:

$$\text{Top}_k^{\mathcal{C}}(q) = \arg \max_{e \in \mathcal{E}, \mathcal{T}_e^+ \cap \mathcal{C} \neq \emptyset} \text{sim}(q, t_e)$$

where $\mathcal{T}_e^+$ denotes the direct types of entity e together with all superclasses reachable through `rdfs:subClassOf`.

The goal is to ensure that all retrieved entities satisfy the ontological constraint $\mathcal{C}$ while preserving as much semantic relevance as possible. This formulation highlights the key challenge addressed in this work: integrating symbolic ontology constraints into vector-based retrieval without sacrificing efficiency.

IV. METHOD

We consider two approaches for integrating ontology constraints into vector retrieval, as shown in Fig. 1: (i) post-hoc filtering, in which constraints are applied after similarity-based retrieval, and (ii) pre-hoc filtering, in which constraints are enforced prior to retrieval by restricting the candidate set.

Post-hoc filtering represents a common approach in existing systems, where a vector database returns the top- k nearest

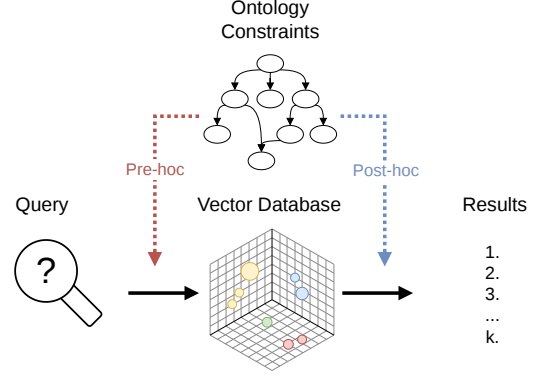


Fig. 1. Ontology-constrained vector retrieval pipeline. A conventional approach applies structured constraints as a post-hoc filter after vector retrieval (blue line). This work implements a pre-hoc filtering approach, using ontology-derived payload fields to restrict the candidate set before similarity ranking (red line).

neighbors and structured constraints are subsequently applied to remove invalid results. If insufficient valid results are obtained, additional candidates must be retrieved and filtered iteratively.

In contrast, pre-hoc filtering incorporates ontology constraints directly into the retrieval process by restricting the candidate set before similarity ranking. This ensures that all returned entities satisfy the specified constraints by construction, assuming the filtered candidate set contains sufficient matching entities.

This implementation proceeds in four stages. First, we parse each ontology-backed dataset from its RDF/OWL source files using `rdflib`. Second, we extract typed ABox entities where available and compute their transitive superclass closure using the ontology’s `rdfs:subClassOf` hierarchy. An entity satisfies a query constraint when its type closure intersects the query’s target class set. Third, we construct an *entity card* for each indexed entity by combining its label, ontological type information, and selected domain-specific attributes. These entity cards are then embedded into dense vector representations using a pretrained sentence-transformer model.

Finally, the entity embeddings are indexed in Qdrant, a vector database supporting approximate nearest-neighbor search with structured payload filtering over stored metadata fields. For each entity, we store both its vector representation and associated ontology-derived payload fields, including:

- `iri`: the canonical ontology identifier,
- `types`: direct class assertions, and
- `types_closure`: direct types plus all superclasses reachable via `rdfs:subClassOf`.

At query time, the natural language query is embedded using the same sentence-transformer model. A corresponding ontology-derived type constraint is then converted into a Qdrant payload filter over `types_closure`. Qdrant applies this filter before similarity ranking, so that nearest-neighbor

search is performed only over candidates satisfying the stored ontology-derived type constraint.

This design yields a practical hybrid retrieval architecture in which symbolic ontology structure enforces type-level semantic constraints, while vector similarity provides flexible semantic ranking. Because the approach uses standard payload filtering, it requires no modification to the underlying vector database, enabling straightforward integration with existing systems.

V. EXPERIMENTAL SETUP

A. Datasets and Ontologies

We evaluate ontology-constrained vector retrieval across multiple ontology-backed datasets. The primary case study uses NASA’s Air Traffic Management ontology (ATMONTTO) and associated National Airspace System data [16]. The source ontology consists of 29 Turtle files containing approximately 2.93 million RDF triples, spanning both schema-level ontology structure (TBox) and large-scale instance data (ABox). ATMONTTO provides a useful testbed because many entity types are semantically related in natural language but ontologically distinct, such as airports, navigation fixes, route segments, and weather products.

We also evaluate the same retrieval pipeline on the Brick ontology, a building-domain ontology for representing equipment, locations, points, and relationships in built environments [17], [18]. Brick provides a substantially different domain structure from ATMONTTO, allowing us to test whether ontology-constrained retrieval generalizes beyond aviation-specific terminology.

Finally, we have constructed an additional DBpedia-based benchmark [19], [20] focused on U.S. civic geography, including entities such as cities, towns, villages, universities, schools, religious institutions, libraries, and other public institutions. This benchmark is designed to evaluate ontology-constrained retrieval in a broader open-domain setting where related entity types frequently overlap in natural-language descriptions.

B. Entity Extraction and Representation

For each ontology-backed dataset, we parse the available RDF/OWL files using `rdflib` and extract typed ABox entities for indexing. An entity is included if it has at least one explicit ontology type and sufficient textual or attribute information to construct a retrieval representation. For each indexed entity, we store its canonical identifier, direct type assertions, and ontology-derived type information.

ATMONTTO serves as our primary enriched case study. In the current prototype, we index 36,655 ATMONTTO entities drawn from six major type families: `ATM#IntersectionFix`, `ATM#LatLonFix`, `ATM#AirspaceRouteSegment`, `NAS#InternationalAirport`, `data#METARreport`, and `data#TAFmeteorologicalCondition`.

For each entity, we compute its transitive superclass closure using the ontology’s `rdfs:subClassOf` graph. This allows retrieval constraints to match both explicitly asserted classes

and inherited superclass categories. We then construct a textual entity description, or entity card, by combining the entity label, type hierarchy, and selected domain-specific attributes. For ATMONTTO, these attributes include coordinates, airport names and codes, fix identifiers, route segment identifiers, and selected meteorological fields when available. For Brick and DBpedia, the same procedure is used, with entity-card attributes adapted to the available fields and vocabulary of each dataset.

C. Embeddings and Vector Index

Entity descriptions are embedded using pretrained sentence-transformer models. In our experiments, we evaluate two embedding models: `all-MiniLM-L6-v2` (384-dimensional) and `bge-large-en-v1.5` (1024-dimensional). These models differ substantially in embedding dimensionality and capacity, allowing us to assess whether the observed effects depend on the underlying representation. For each model, embeddings are computed independently and indexed in Qdrant under identical conditions. These vectors are indexed in a local Qdrant instance deployed via Docker.

Each indexed point in Qdrant stores:

- an embedding vector (dimension depends on the model),
- an entity identifier (`iri`),
- direct ontology types (`types`), and
- transitive type closure (`types_closure`).

D. Retrieval Conditions

We evaluate three retrieval modes:

- **Unconstrained retrieval:** standard top- k nearest-neighbor search over the full index.
- **Pre-hoc ontology-constrained retrieval:** nearest-neighbor search restricted before ranking using a Qdrant payload filter over `types_closure`.
- **Post-hoc ontology-constrained retrieval:** standard nearest-neighbor search over the full index followed by filtering of the returned candidates using the same ontology constraint.

We use a set of natural language queries for each evaluated ontology. For ATMONTTO, these queries reflect realistic air traffic information needs, including navigation fixes, route segments, airport entities, and weather products. Each query is paired with a target ontology class or class family representing valid result types.

All experiments use $k = 5$. We report:

- **valid@5:** fraction of the top-5 retrieved items that satisfy the ontology constraint,
- **success@5:** fraction of queries for which at least one valid result appears in the top-5,
- **score:** mean similarity score of the top-5 results,
- **examined:** average number of candidates evaluated during retrieval (for post-hoc methods),
- **query time:** average query latency.

For each query, we define a target ontology class or class family representing the valid result type. A retrieved entity is

TABLE I
AVERAGE RETRIEVAL PERFORMANCE ACROSS DATASETS FOR EACH EMBEDDING MODEL.

Method	Cap	valid@5	success@5	score	time (s)
all-MiniLM-L6-v2					
Unconstrained	5	0.6570	0.5095	0.5554	0.0084
Pre-hoc	5	1.0000	1.0000	<u>0.5387</u>	0.0178
Post-hoc	25	0.7881	0.7590	0.4415	0.0133
Post-hoc	100	0.8967	0.8603	0.4945	0.0228
Post-hoc	200	0.9289	0.8962	0.5160	0.0300
Post-hoc	400	0.9454	0.9254	0.5228	0.0430
Post-hoc	2000	<u>0.9734</u>	<u>0.9654</u>	0.5312	0.1215
bge-large-en-v1.5					
Unconstrained	5	0.7453	0.5790	0.7011	0.0087
Pre-hoc	5	1.0000	1.0000	0.6972	0.0207
Post-hoc	25	0.8947	0.8724	0.6453	0.0141
Post-hoc	100	0.9388	0.9375	0.6574	0.0182
Post-hoc	200	0.9521	0.9521	0.6573	0.0238
Post-hoc	400	0.9534	0.9521	0.6638	0.0331
Post-hoc	2000	<u>0.9848</u>	<u>0.9787</u>	0.6907	0.0853

counted as valid if its ontology-derived type closure contains the target class, or one of the target classes in the case of class-family constraints. For unconstrained retrieval, validity is evaluated after retrieving the top-5 nearest neighbors from the full vector index. For pre-hoc retrieval, the ontology constraint is applied before similarity ranking using a payload filter over `types_closure`; therefore, all returned entities satisfy the constraint by construction, assuming the index contains sufficient matching candidates. For post-hoc retrieval, the system first retrieves a larger candidate set from the unconstrained index, filters those candidates by the same ontology constraint, and then reports the first five valid results if available.

We compute **valid@5** as the fraction of the five reported results that satisfy the target ontology constraint, averaged across queries. We compute **success@5** as the fraction of queries for which at least one valid entity appears among the five reported results. If fewer than five valid entities are found under post-hoc filtering, missing positions are treated as invalid for valid@5, and the query is counted as unsuccessful for success@5 only if no valid entity is returned.

VI. RESULTS

Table I summarizes the results per the embedding models, while Table II shows retrieval performance across all datasets and embedding models. Fig. 2 shows valid@5 scores for each dataset across retrieval candidate limits averaged over embedding models.

Across both embedding models, ontology-constrained (pre-hoc) retrieval achieves perfect type validity, improving valid@5 from 0.6570 (MiniLM) and 0.7453 (BGE) to 1.00. In contrast, unconstrained retrieval frequently returns semantically similar but ontologically invalid entities, particularly for queries where type constraints are not strongly implied by surface-level semantics.

The behavior of post-hoc filtering differs between models. For all-MiniLM-L6-v2, post-hoc retrieval improves with increasing candidate caps but fails to reach full validity,

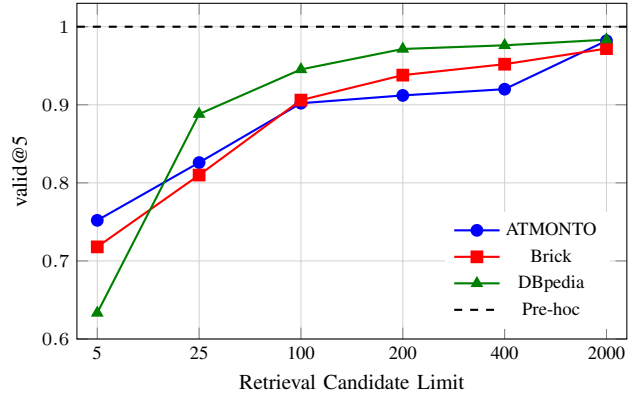


Fig. 2. Average valid@5 across embedding models for unconstrained retrieval and post-hoc filtering at increasing candidate caps. The dashed line shows pre-hoc filtering.

plateauing at valid@5 = 0.9734 even when examining an average of 228 (and up to 2000) candidates per query. This indicates that increased candidate exploration does not fully resolve type inconsistency for weaker embedding models. For bge-large-en-v1.5, post-hoc retrieval reaches valid@5 = 0.9848 at a cap of 2000, still falling short of perfect validity, while incurring a latency approximately $4.1 \times$ higher than pre-hoc retrieval.

Across both models, post-hoc filtering requires substantially greater candidate exploration to approach the validity achieved by pre-hoc constraint enforcement. In contrast, pre-hoc retrieval guarantees perfect validity while examining only $k = 5$ candidates per query, yielding a consistent and predictable computational profile.

While constrained retrieval slightly reduces average similarity scores, this reflects the exclusion of semantically strong but ontologically invalid matches rather than a degradation in retrieval quality.

Overall, these results demonstrate that enforcing ontology constraints during retrieval provides a more efficient and reliable mechanism for ensuring satisfaction of the specified ontology-derived type constraint than post-hoc filtering, particularly in settings where type constraints are not easily inferred from natural language alone.

VII. ANALYSIS

We analyze the behavior of unconstrained and ontology-constrained retrieval to better understand the observed performance differences.

A. Failure Modes of Unconstrained Retrieval

Unconstrained vector retrieval ranks entities solely by embedding similarity, commonly computed between dense query and document representations [21], [22]. Such embeddings are designed to capture semantic and contextual relatedness, but similarity alone does not enforce task-specific structural constraints. As a result, retrieval may prioritize entities that are strongly associated with prominent query terms even when

those entities are ontologically inappropriate for the intended query.

For example, ATMONTO queries containing airport identifiers such as “JFK” tend to retrieve airport entities regardless of the intended query type. This occurs because embedding models strongly associate such tokens with their most frequent semantic contexts (e.g., airports), rather than with structurally constrained concepts such as route segments or navigation fixes.

This explains the observed failure in route segment queries, where unconstrained retrieval returns only airport entities, yielding zero valid results. Importantly, this failure mode is observed across embedding models, indicating that it arises from the behavior of similarity-based retrieval rather than properties of a specific embedding space.

B. Effect of Ontology Constraints

Ontology constraints alter the retrieval process by restricting the candidate set prior to similarity ranking. Formally, retrieval is performed over a filtered subset of entities satisfying the ontology constraint, rather than over the full entity set.

This has two important effects. First, it guarantees semantic validity by eliminating all entities that violate the specified type constraint. Second, it modifies the ranking distribution by removing high-scoring but invalid candidates, allowing lower-scoring but valid entities to surface.

This mechanism explains the observed tradeoff between validity and similarity: ontology-constrained retrieval eliminates invalid candidates while allowing lower-scoring but structurally correct entities to surface.

C. Qualitative Example

The query “*arrival route segment near JFK*” illustrates the difference between unconstrained and ontology-constrained retrieval.

Unconstrained retrieval (top-5):

- JFK Airport
- EFKJ Airport
- EFKT Airport
- EFKA Airport
- EFKK Airport

Ontology-constrained retrieval (top-5):

- J518 Subpath07620120908_01
- NKS174-201407150027_AR16
- NKS174-201407150027_J191
- JBU582-201407150007_J37
- JBU576-201407150034_J121

Under unconstrained retrieval, all returned entities are airports, reflecting strong semantic association with the token “JFK.” None satisfy the intended query type, yielding zero valid results. In contrast, ontology-constrained retrieval returns only valid `AirspaceRouteSegment` entities.

This example highlights the core limitation of embedding-based retrieval: semantic proximity alone does not enforce

structural correctness. This limitation persists across embedding models, reinforcing the need for explicit constraint mechanisms.

VIII. DISCUSSION

These findings reinforce a key limitation of embedding-based retrieval: semantic similarity alone does not guarantee structurally valid results. This limitation persists across embedding models, indicating that it is intrinsic to similarity-based retrieval rather than a limitation of a specific representation.

By enforcing ontology-derived constraints prior to retrieval, our approach ensures type-level semantic validity with respect to the specified ontology class constraints while preserving meaningful ranking behavior. The effectiveness of this approach stems from restricting the candidate set before similarity ranking, preventing high-scoring but invalid entities from dominating the results. The consistency of this improvement across embedding models suggests that ontology-constrained retrieval provides a robust and model-agnostic mechanism for integrating symbolic structure into vector search systems.

This pre-hoc approach also contrasts sharply with post-hoc filtering. While post-hoc methods provide partial improvements at low candidate caps, they require substantially greater candidate exploration to achieve full validity, and may fail to reach it entirely depending on the embedding model. Even when full validity is achieved, it typically requires examining an order of magnitude more candidates than pre-hoc retrieval. Pre-hoc filtering, by comparison, enforces validity by construction while maintaining efficient and predictable retrieval.

A further observation is the interaction between embedding dimensionality and constraint enforcement. Higher-capacity embeddings (e.g., `bge-large-en-v1.5`) improve unconstrained and post-hoc retrieval performance, reducing the frequency of type violations and enabling post-hoc filtering to approach but not consistently achieve full validity. However, this improvement comes at increased computational cost due to higher-dimensional vector operations. Notably, combining pre-hoc constraint enforcement with lower-dimensional embeddings (e.g., `all-MiniLM-L6-v2`) yields the most efficient configuration that still guarantees perfect validity. This highlights a practical tradeoff: stronger embeddings can partially compensate for the absence of constraints, but pre-hoc filtering enables smaller, more efficient models to achieve strictly correct retrieval with lower overall cost.

Several limitations of the current system merit acknowledgment. First, the evaluation is conducted over three ontologies from distinct but structured domains. Whether these finds generalize to loosely structured or open-domain ontologies remains an open question. Second, the system does not incorporate explicit spatial reasoning and thus queries involving geographic relationships (e.g., “near JFK”) rely solely on embedding similarity, which may yield results that satisfy type constraints but lack true geographic relevance. Third, the effectiveness of constraints depends on their alignment with query intent. For example, restricting airport queries to the class `InternationalAirport` can produce semantically incorrect

results when the intended scope differs. This highlights the importance of accurate query-to-ontology mapping in constrained retrieval systems, and training over the correct usage by end-users.

IX. CONCLUSION

This work demonstrates that ontology-constrained vector retrieval provides a practical and effective mechanism for enforcing semantic correctness in embedding-based search systems within structured, ontology-backed domains. By integrating ontology-derived constraints directly into the retrieval process, our pre-hoc approach achieves perfect type validity by construction while maintaining efficient and predictable query performance.

Across three ontologies spanning distinct domains – aviation (ATMONT), building systems (Brick), and civic geography (DBpedia) – and two embedding models, our results consistently show that unconstrained retrieval frequently returns semantically related but structurally invalid results, while post-hoc filtering requires substantially greater candidate exploration to achieve comparable validity, and may fail to reach it entirely depending on factors like embedding model and dataset.

These findings suggest that, in domain-specific settings where entities are typed according to a formal ontology and type requirements are not reliably inferred from natural language alone, semantic constraints should be treated as a first-class component of vector retrieval. Whether these benefits extend to more open-domain or loosely structured settings remains an open question for future work. More broadly, this work contributes to a growing body of evidence for hybrid neuro-symbolic approaches that combine geometric similarity with formal structure to improve the reliability and controllability of modern retrieval systems.

X. FUTURE WORK

Our current system demonstrates that ontology-constrained vector retrieval significantly improves type validity over unconstrained semantic search. To extend this work, we consider more expressive and formally-constrained approaches as natural extensions of this work.

A. Formal Ontology Reasoning via Description Logics

A natural next step for our work would be to leverage a Description Logics (DL) reasoner. The ATM ontology is grounded in OWL, whose semantics are defined by these description logics. Integrating a DL reasoner would enable implicit type materialization: inferring that an entity belongs to a class based on its properties, even when that membership is not explicitly asserted. This would enrich our types closure beyond what structural graph traversal captures, reducing false negatives in constrained retrieval.

B. Tableau-Based Consistency Verification

Another large goal of our project is verification of retrieval. Tableau-based reasoning could serve as this post-retrieval verification step: after candidate entities are retrieved via vector search, a reasoner would verify that each candidate is logically consistent with the query constraints and the broader ontology before returning results. This would allow the system to reject entities that satisfy type filters but violate deeper ontological axioms.

C. Consistent Sub-World Reasoning

Our dataset comes with no guarantee of being absolutely correct. As a real-world aviation knowledge graph, there could be inconsistent data, especially when concerning time-stamped observations. When global consistency in the ontology cannot be guaranteed, reasoning can be difficult or impossible. Such “ontology repairs,” [23] that allow the system to identify the most coherent available slice of the knowledge graph, have been studied before using techniques such as axiom weakening [24]. We could incorporate these techniques to increase the reliability of retrieval in our setting.

D. Neuro-Symbolic Retrieval Architecture

These directions contribute to the bigger picture of a unified *neuro-symbolic* retrieval architecture. Vector search would serve as the candidate generator, and formal reasoning would handle consistency.

These approaches are growing in popularity, and have been explored explicitly with “domain ontologies [playing] the role of symbolic structures” [25].

The evaluation framework introduced in this work, measuring $\text{valid}@k$, provides a natural baseline against which neuro-symbolic hybrid systems can be benchmarked.

E. Ontology Learning

The current framework assumes that a well-formed ontology is available and that query-to-class mappings are specified explicitly. In practice, however, ontologies may be incomplete, unavailable, or difficult to derive and construct. Ontology learning is the automated or semi-automated induction of ontological structure from data, and presents a natural extension of this work [26]. Integrating ontology learning techniques could allow the system to refine type constraints from the underlying data, reducing manual overhead of ontology construction and broadening the applicability to domains where formal ontologies do not yet exist. This would move the framework toward a more self-contained pipeline capable of operating in less structured settings.

APPENDIX A
COMPLETE RETRIEVAL RESULTS

TABLE II: Master comparison of retrieval strategies across ontologies and embedding models.

Ontology	Retrieval Limit	valid@5	success@5	time (s)	examined
ATMONT0, all-MiniLM-L6-v2					
Unconstrained	5	0.6560	0.5600	0.0081	5
Pre-hoc	5	1.0000	1.0000	0.0178	5
Post-hoc	25	0.7280	0.6800	0.0102	25
Post-hoc	100	0.8640	0.8400	0.0218	45
Post-hoc	200	0.8840	0.8600	0.0303	61
Post-hoc	400	0.8960	0.8600	0.0509	89
Post-hoc	2000	0.9640	0.9400	0.1962	228
ATMONT0, bge-large-en-v1.5					
Unconstrained	5	0.8480	0.7800	0.0068	5
Pre-hoc	5	1.0000	1.0000	0.0218	5
Post-hoc	25	0.9240	0.9200	0.0111	25
Post-hoc	100	0.9400	0.9400	0.0147	31
Post-hoc	200	0.9400	0.9400	0.0202	37
Post-hoc	400	0.9440	0.9400	0.0294	49
Post-hoc	2000	1.0000	1.0000	0.0355	62
Brick, all-MiniLM-L6-v2					
Unconstrained	5	0.6720	0.5400	0.0073	5
Pre-hoc	5	1.0000	1.0000	0.0138	5
Post-hoc	25	0.7600	0.7400	0.0109	25
Post-hoc	100	0.8880	0.8600	0.0214	42
Post-hoc	200	0.9360	0.9000	0.0281	53
Post-hoc	400	0.9640	0.9400	0.0431	70
Post-hoc	2000	0.9800	0.9800	0.0811	104
Brick, bge-large-en-v1.5					
Unconstrained	5	0.7640	0.6000	0.0081	5
Pre-hoc	5	1.0000	1.0000	0.0151	5
Post-hoc	25	0.8600	0.8400	0.0139	25
Post-hoc	100	0.9240	0.9200	0.0185	36
Post-hoc	200	0.9400	0.9400	0.0249	43
Post-hoc	400	0.9400	0.9400	0.0379	55
Post-hoc	2000	0.9640	0.9600	0.1405	142
DBpedia US Civic Geography, all-MiniLM-L6-v2					
Unconstrained	5	0.6429	0.4286	0.0097	5
Pre-hoc	5	1.0000	1.0000	0.0218	5
Post-hoc	25	0.8762	0.8571	0.0188	25.00
Post-hoc	100	0.9381	0.8810	0.0252	35.71
Post-hoc	200	0.9667	0.9286	0.0316	44.64
Post-hoc	400	0.9762	0.9762	0.0349	51.19
Post-hoc	2000	0.9762	0.9762	0.0873	89.29
DBpedia US Civic Geography, bge-large-en-v1.5					
Unconstrained	5	0.6238	0.3571	0.0113	5
Pre-hoc	5	1.0000	1.0000	0.0252	5
Post-hoc	25	0.9000	0.8571	0.0174	25.00
Post-hoc	100	0.9524	0.9524	0.0215	32.74

Continued on next page

Ontology	Retrieval Limit	valid@5	success@5	time (s)	examined
Post-hoc	200	0.9762	0.9762	0.0262	37.50
Post-hoc	400	0.9762	0.9762	0.0320	42.26
Post-hoc	2000	0.9905	0.9762	0.0798	80.36

REFERENCES

- [1] J. J. Pan, J. Li, Z. Zhao, and Y. Tang, "A survey of vector database management systems," *arXiv preprint arXiv:2310.14021*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.14021>
- [2] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *arXiv preprint arXiv:2309.11322*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.11322>
- [3] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," *ArXiv*, vol. abs/2602.11443, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285540862>
- [4] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, and Y. Zhang, "Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters," in *Proceedings of the ACM Web Conference*, 2023, pp. 3406–3416.
- [5] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "Acorn: Performant and predicate-agnostic search over vector embeddings and structured data," *Proceedings of the ACM on Management of Data*, vol. 2, no. 3, pp. 1–27, 2024.
- [6] B. Smith, "Ontology," *The Blackwell Guide to the Philosophy of Computing and Information*, pp. 153–166, 2004.
- [7] I. Niles and A. Pease, "Towards a standard upper ontology," in *Proceedings of the International Conference on Formal Ontology in Information Systems - Volume 2001*, ser. FOIS '01. New York, NY, USA: Association for Computing Machinery, 2001, p. 2–9. [Online]. Available: <https://doi.org/10.1145/505168.505170>
- [8] T. Neumann and G. Weikum, "Rdf-3x: A risc-style engine for rdf," in *Proceedings of the VLDB Endowment*, vol. 1, no. 1, 2008, pp. 647–659.
- [9] R. Cyganiak, D. Wood, and M. Lanthaler, "Rdf 1.1 concepts and abstract syntax," <https://www.w3.org/TR/rdf11-concepts/>, 2014, w3C Recommendation.
- [10] W. O. W. Group, "Owl 2 web ontology language: Document overview," <https://www.w3.org/TR/owl2-overview/>, 2012, w3C Recommendation.
- [11] W3C, "SPARQL 1.1 Query Language," <https://www.w3.org/TR/sparql11-query/>, 2013, w3C Recommendation.
- [12] —, "SPARQL 1.1 Entailment Regimes," <https://www.w3.org/TR/sparql11-entailment/>, 2013, w3C Recommendation.
- [13] L. Ma, R. Zhang, Y. Han, S. Yu, Z. Wang, Z. Ning, J. Zhang, P. Xu, P. Li, W. Ju *et al.*, "A comprehensive survey on vector database: Storage and retrieval technique, challenge," *arXiv preprint arXiv:2310.11703*, 2023.
- [14] A. Gali, C. X. Chen, K. T. Claypool, and R. Uceda-Sosa, "From ontology to relational databases," in *International Conference on Conceptual Modeling*. Springer, 2004, pp. 278–289.
- [15] R. W. Group, Feb 2014. [Online]. Available: <https://www.w3.org/RDF/>
- [16] NASA, "Air traffic management ontology for the national airspace system," NASA, Tech. Rep. NASA Technical Report 20170006095, 2017. [Online]. Available: <https://ntrs.nasa.gov/citations/20170006095>
- [17] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Berges, D. Culler, R. K. Gupta, M. B. Kjergaard, M. Srivastava, and K. Whitehouse, "Brick: Towards a unified metadata schema for buildings," in *Proceedings of the 3rd ACM International Conference on Systems for Energy-Efficient Built Environments*. ACM, 2016, pp. 41–50.
- [18] Brick Schema, "Brick ontology," <https://brickschema.org/>, accessed: 2026-04-30.
- [19] P. N. Mendes, M. Jakob, A. García-Silva, and C. Bizer, "Dbpedia: A multilingual cross-domain knowledge base," in *Proceedings of the Eighth International Conference on Language Resources and Evaluation*, 2012, pp. 1813–1817.
- [20] DBpedia Association, "Dbpedia ontology," <https://www.dbpedia.org/resources/ontology/>, accessed: 2026-04-30.
- [21] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2020, pp. 6769–6781.
- [22] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992.
- [23] C. Meilicke, H. Stuckenschmidt, and A. Tamin, "Repairing ontology mappings," in *AAAI*, vol. 3, no. 2.1, 2007, p. 6.
- [24] N. Troquard, R. Confalonieri, P. Galliani, R. Penaloza, D. Porello, and O. Kutz, "Repairing ontologies via axiom weakening," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [25] A. Smirnov, A. Ponomarev, and A. Agafonov, "Ontology-based neuro-symbolic ai: effects on prediction quality and explainability," *IEEE Access*, vol. 12, pp. 156 609–156 626, 2024.
- [26] A. Maedche and S. Staab, "Learning ontologies for the semantic web," in *Proceedings of the Second International Conference on Semantic Web - Volume 40*, ser. SemWeb'01. Aachen, DEU: CEUR-WS.org, 2001, p. 51–60.